



Introducción a la ciencia de datos

Capítulo 3: Almacenamiento y lectura de datos

Mg. J. Eduardo Gamboa U.

2026-09-08

Iniciando un proyecto en ciencia de datos

- ▶ Primer paso: definir los objetivos = $f(\text{problema})$
- ▶ ¿Qué? ¿Por qué? ¿Cómo?
- ▶ ¿Cuáles son las expectativas de las partes involucradas (alta dirección, desarrolladores, analistas, usuarios del sistema, clientes)?
- ▶ ¿Cómo el proyecto va a cambiar el negocio?
- ▶ ¿Cómo se usarán los resultados?
- ▶ Cronograma del proyecto con acuerdos de entregables (fechas, responsables, porcentaje de avance, asignación de recursos, etc)

Almacenamiento en Big Data

Sistemas de almacenamiento

01

Repositorios

- Base de datos
- Data mart
- Data warehouse
- Data lake

02

Tecnologías

- Apache Hadoop
- MongoDB
- Cassandra
- Apache HBase

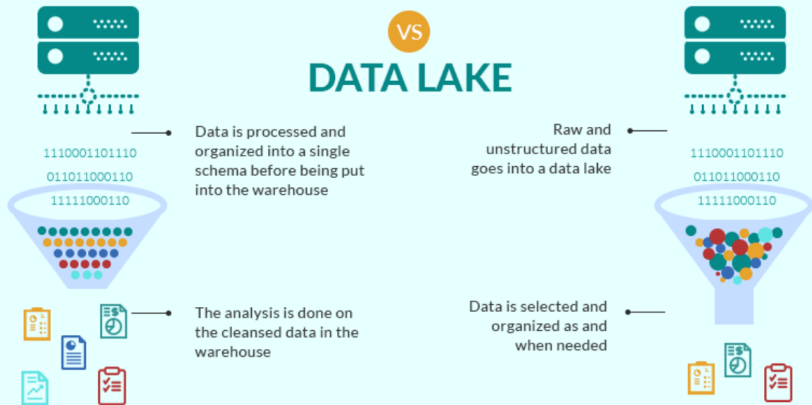
Repositorios

1. **Data lake:** Es un medio de almacenamiento de data estructurada , semiestructurada y no estructurada (en su formato original). Facilita la organización y gestión de grandes volúmenes de datos provenientes de diversas fuentes, como **CRM**, **ERP**, Redes sociales, **IoT** y/o sensores.
2. **Data warehouse (DWH):** Es un sistema centralizado de almacenamiento que integra datos de múltiples fuentes, generalmente estructurados. Aunque su enfoque principal es el manejo de datos estructurados, también puede procesar datos no estructurados (utilizando **ETL**).
3. **Data mart:** Es un almacén de datos diseñado para un área específica dentro de una organización. Usualmente, es un subconjunto de un Data Warehouse, optimizado para consultas y análisis en un dominio particular.
4. **Database (base de datos):** Es una colección organizada de datos estructurados en tablas (bases de datos relacionales **SQL**) o en formatos flexibles y no estructurados (noSQL)

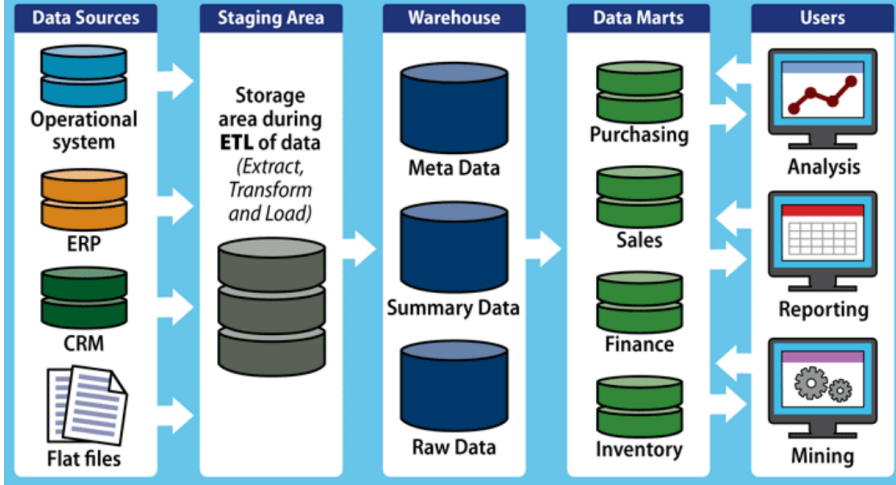
DATA WAREHOUSE

VS

DATA LAKE



Data Warehouse Architecture

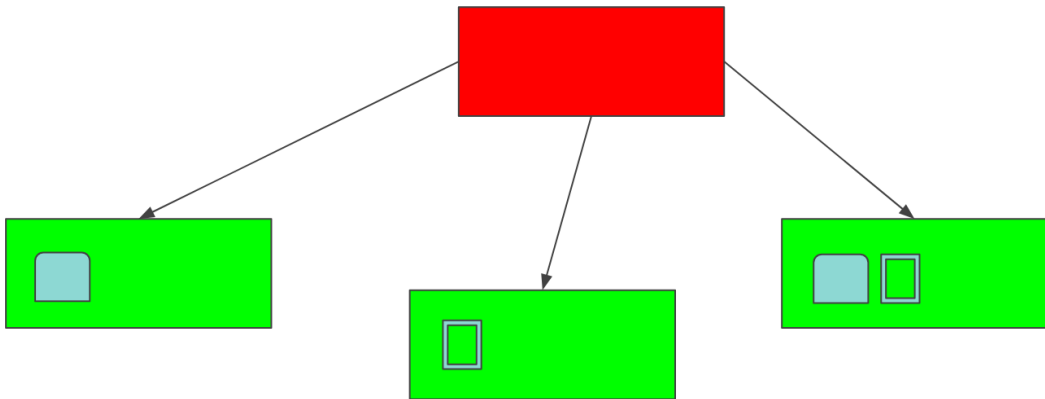


Tecnologías

Apache Hadoop



- ▶ Es un entorno de trabajo (ecosistema) de software de código abierto que permite el almacenamiento de datos en clusters de servidores.
- ▶ Un cluster está conformado por un nodo o máquina maestra (primaria) y múltiples nodos secundarios (ex - esclavos).
- ▶ Hadoop fue creado por Doug Cutting y Mike Cafarella en 2006, inspirado por los trabajos de Google sobre el procesamiento y almacenamiento de datos a gran escala.
- ▶ Fue lanzado oficialmente por la Apache Software Foundation en 2011.



- ▶ Actualmente es gestionado y mantenido por la Apache Software Foundation (ASF).
- ▶ Licenciado bajo la Licencia Apache 2.0, permitiendo así su uso, modificación y distribución.
- ▶ Hadoop es altamente escalable, lo que permite añadir o quitar nodos según sea necesario.
- ▶ También ofrece tolerancia a fallos replicando los datos en múltiples nodos del cluster.

- ▶ Almacena todo tipo de datos: estructurado y no estructurado, y metadatos
- ▶ Sus principales componentes son **HDFS** y MapReduce.
- ▶ Diversas empresas usan servicios y soluciones comerciales basadas en Hadoop, ofrecidos por Cloudera, IBM, Amazon Web Services, Microsoft, Google, etc.
- ▶ Empresas que usan Hadoop: Adobe, Alibaba, EBay, Facebook, Hulu, IBM, LinkedIn, MercadoLibre, The New York Times, Spotify, Telefónica, Twitter, Yahoo, etc. Más empresas que usan Hadoop aquí.
- ▶ No es eficiente para tareas iterativas y/o interactivas (a menos que se use en conjunto con Apache Spark)

mongoDB

- ▶ Viene del término inglés humongous
- ▶ Fue lanzado en el año 2009 por 10gen (ahora MongoDB Inc.)
- ▶ Maneja sistema de base de datos **no SQL** (no relacional)
- ▶ Almacena los datos en una extensión del formato **JSON**.
- ▶ Es recomendable usarlo cuando nuestros datos están almacenados en la nube, varían mucho (en tiempo real) y varios usuarios acceden a ella.
- ▶ Empresas que usan mongoDB: The Weather Channel, Adobe, Nokia, eBay, Google, Facebook, PayPal, Telefónica. Más empresas aquí.

Lectura de datos estructurados

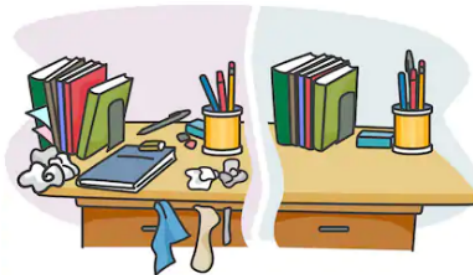
Archivos de datos

Los datos pueden estar contenidos en archivos de distintos formatos, entre los que se tiene los más comunes:

1. Archivo de texto plano: su contenido es netamente textual, sin ningún formato. Lleva la extensión .txt y puede ser creado y/o editado en Bloc de notas.
2. Archivo de valores separados por comas: permite representar datos en formato tabular, en el que las columnas son separadas por comas (o puntos y comas, dependiendo del país y/o la configuración de la computadora). Lleva la extensión .csv y puede ser creado y/o editado en programas como Bloc de notas o Excel.
3. Archivo de hoja de cálculo: permite almacenar los datos en celdas, dispuestas en filas y columnas. La extensión común es .xlsx o .xls

Antes de iniciar con la lectura de datos...

...debemos entender y ordenar nuestro espacio de trabajo



¿Cómo RStudio accede al archivo que contiene nuestro conjunto de datos? ¿Dónde “vive” nuestro código de análisis de datos?

Ejecute las líneas de código:

```
getwd()  
list.files()
```

Por defecto, R ejecutará todos nuestros análisis en esta carpeta. ¿Cómo cambiamos esa ruta?

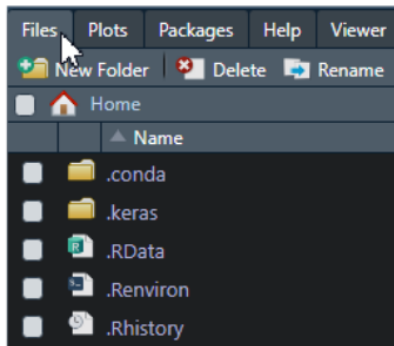
1. Usar la función `setwd(...)`
2. Usar `proyecto(s)`: opción reproducible

Entorno de trabajo

¿Quiénes son los “vecinos” de nuestro código de análisis de datos?

Puedes verlo en la pestaña Files del panel inferior derecho.

Se listan todos los archivos que conforman la carpeta que obtuvimos con la función `getwd()` o la que fijamos con `setwd(...)`.



Reproducibilidad

La reproducibilidad es importante. Si tú ejecutas un código y se lo compartes a otra persona, debe obtener el mismo resultado, sin necesidad de realizar cambios en el código. ¿Qué sucede si un código comienza con `setwd("C:/Users/Usuario 1")`?

Proyectos en RStudio

Permite manejar el entorno de trabajo de manera ordenada. Todos los archivos quedan empaquetados en una carpeta de trabajo.

¿Cómo crear un proyecto en RStudio?

1. Crear una carpeta de trabajo, la cual contiene o contendrá todos los archivos necesarios para el análisis.
2. Click en File → New Project o hacer click en los íconos que ejecutan esa misma acción
3. Elegir “Existing Directory” y luego en “Browse” buscar la carpeta de trabajo

Lectura de datos

Utilizaremos distintas funciones, como por ejemplo `import`, `read.table`, `read_csv2` o `read_xlsx`, pero antes de ello necesitaremos instalar los paquetes `rio`, `readr` y `readxl`.

```
install.packages("rio")  
install.packages("readr")  
install.packages("readxl")
```

Luego, los cargamos:

```
library(rio)  
library(readr)  
library(readxl)
```

`import`

```
datos <- import(file, which, ...)
```

- ▶ `file`: Argumento character obligatorio. Corresponde al archivo de datos que será leído. Va entre comillas y debe incluir su extensión, por ejemplo "datos301.txt"
- ▶ `which`: Argumento opcional utilizado cuando un archivo contiene más de un conjunto de datos. En archivos Excel permite indicar la hoja que se desea importar, mediante su nombre o posición. Por ejemplo, `which = "Hoja2"` o `which = 2`.

Ejemplo 1

El archivo Cap3 - INEI - 001.txt contiene datos sobre la población de 0 a 5 años estimada por el INEI para el año 2021 en el distrito de Bellavista, mientras que el archivo Cap3 - INEI - Diccionario.xlsx proporciona el diccionario de datos.

```
datos1a = import(file = "Cap3 - INEI - 001.txt")
```

Verificando que los datos han sido importados correctamente.

Uso de la función head

```
head(datos1a)
```

	ubigeo_reniec	ubigeo_inei	Departamento	Provincia	Distrito	Edad_Anio	Sexo
1	240102	70102	CALLAO	CALLAO	BELLAVISTA	5	F
2	240102	70102	CALLAO	CALLAO	BELLAVISTA	5	M
3	240102	70102	CALLAO	CALLAO	BELLAVISTA	4	F
4	240102	70102	CALLAO	CALLAO	BELLAVISTA	4	M
5	240102	70102	CALLAO	CALLAO	BELLAVISTA	3	F
6	240102	70102	CALLAO	CALLAO	BELLAVISTA	3	M

Cantidad

1	420
2	403
3	417
4	398
5	429
6	383

```
head(datos1a, n = 3)
```

	ubigeo_reniec	ubigeo_inei	Departamento	Provincia	Distrito	Edad_Anio	Sexo
1	240102	70102	CALLAO	CALLAO	BELLAVISTA	5	F
2	240102	70102	CALLAO	CALLAO	BELLAVISTA	5	M
3	240102	70102	CALLAO	CALLAO	BELLAVISTA	4	F

Cantidad

1	420
2	403
3	417

Uso de la función tail

```
tail(datos1a)
```

	ubigeo_reniec	ubigeo_inei	Departamento	Provincia	Distrito	Edad_Anio	Sexo
7	240102	70102	CALLAO	CALLAO	BELLAVISTA	2	F
8	240102	70102	CALLAO	CALLAO	BELLAVISTA	2	M
9	240102	70102	CALLAO	CALLAO	BELLAVISTA	1	F
10	240102	70102	CALLAO	CALLAO	BELLAVISTA	1	M
11	240102	70102	CALLAO	CALLAO	BELLAVISTA	0	F
12	240102	70102	CALLAO	CALLAO	BELLAVISTA	0	M

Cantidad

7	427
8	381
9	450
10	354
11	392
12	405

```
tail(datos1a, n = 8)
```

	ubigeo_reniec	ubigeo_inei	Departamento	Provincia	Distrito	Edad_Anio	Sexo
5	240102	70102	CALLAO	CALLAO	BELLAVISTA	3	F
6	240102	70102	CALLAO	CALLAO	BELLAVISTA	3	M
7	240102	70102	CALLAO	CALLAO	BELLAVISTA	2	F
8	240102	70102	CALLAO	CALLAO	BELLAVISTA	2	M
9	240102	70102	CALLAO	CALLAO	BELLAVISTA	1	F
10	240102	70102	CALLAO	CALLAO	BELLAVISTA	1	M
11	240102	70102	CALLAO	CALLAO	BELLAVISTA	0	F
12	240102	70102	CALLAO	CALLAO	BELLAVISTA	0	M

Cantidad

5	429
6	383
7	427
8	381
9	450
10	354
11	392
12	405

read.table

```
datos <- read.table(file, header, sep, dec, col.names, row.names,  
                    na.strings, nrows, skip)
```

- ▶ **file**: Argumento character obligatorio. Corresponde al archivo de datos a ser leído. Va entre comillas y debe incluir su extensión
- ▶ **header**: Argumento logical opcional. TRUE si la primera línea contiene los nombres de las variables. Por defecto se asume que es FALSE, a menos que el número de campos en la primera fila sea uno menos que el número de columnas.
- ▶ **sep**: Argumento character opcional. Indica el separador entre columnas. Va entre comillas, por defecto asume que es `sep = " "` (al menos un espacio en blanco).
- ▶ **dec**: Argumento character opcional. Indica el separador de la parte entera y decimal en los números, por defecto asume `dec = "."`

```
datos <- read.table(file, header, sep, dec, col.names, row.names,  
                    na.strings, nrows, skip)
```

- ▶ `col.names`: Argumento de tipo vector character, opcional. Indica el nombre de las columnas.
- ▶ `row.names`: Argumento de tipo múltiple, opcional. Puede ser un vector character que indica el nombre de las filas. También se puede indicar el número de columna contiene estos nombres.
- ▶ `na.strings`: Argumento character opcional. Indica cómo se declaran los valores perdidos (Not Available).
- ▶ `nrows`: Argumento integer opcional. Indica el número máximo de filas a leer. Por defecto es -1, de modo que dicho valor es ignorado (y así lee el archivo completo).
- ▶ `skip`: Argumento integer opcional. Indica el número de filas iniciales del archivo que no serán leídas. Por defecto es 0.

Ejemplo 1 (cont.)

Ejecutar los siguientes códigos para leer el archivo. ¿Realizan la misma acción? ¿En qué se diferencian?

Pista: Usar la función `identical`

```
datos1a = import(file = "Cap3 - INEI - 001.txt")
datos1b = read.table(file = "Cap3 - INEI - 001.txt", header = TRUE)
datos1c = read.table("Cap3 - INEI - 001.txt", TRUE)
datos1d = read.table(header = TRUE, file = "Cap3 - INEI - 001.txt")
```

read_tsv

Una vez instalado el paquete readr, cargarlo:

```
library(readr)
```

```
datos <- read_tsv(file, col_names, na, skip, n_max)
```

- ▶ **file**: Argumento character obligatorio. Corresponde al archivo de datos a ser leído. Va entre comillas. Incluir su extensión (txt), por ejemplos "datos123.txt"
- ▶ **col_names**: Argumento opcional de múltiple tipo. Puede ser logical (TRUE o FALSE) si indica la presencia o ausencia de nombres de columnas (como header en read.table), o puede ser un vector indicando los nombres de las columnas. Por defecto toma el valor TRUE
- ▶ **na**: Argumento de tipo vector character opcional. Indica cómo se declaran los valores perdidos. Por defecto es c("", "NA").

```
datos <- read_tsv(file, col_names, na, skip, n_max)
```

- ▶ skip: Argumento numérico opcional. Indica el número de filas iniciales del archivo que no serán leídas. Por defecto es 0.
- ▶ n_max: Argumento numérico opcional. Indica el número máximo de filas a leer. Por defecto es infinito (Inf).

Ejercicio 1 (cont.)

Volver a leer el archivo Cap3 - INEI - 001.txt utilizando la función `read_tsv`. Tener en cuenta que los argumentos pueden variar en nombre y no son necesariamente los mismos, además de que no son los únicos disponibles. El único argumento obligatorio es `file`.

read.csv

```
datos <- read.csv(file, header, sep, dec, col.names, row.names,  
                  na.strings, nrow, skip)
```

- ▶ **file**: Argumento character obligatorio. Corresponde al archivo de datos a ser leído. Va entre comillas. Incluir su extensión (txt, csv), por ejemplos "datos159.csv"
- ▶ **header**: Argumento logical opcional. TRUE si la primera línea contiene los nombres de las variables. Por defecto se asume que es TRUE.
- ▶ **sep**: Argumento character opcional. Indica el separador entre columnas. Va entre comillas, por defecto asume que es `sep = ","` (coma).
- ▶ **dec**: Argumento character opcional. Indica el separador de la parte entera y decimal en los números, por defecto asume `dec = "."`.

```
datos <- read.csv(file, header, sep, dec, col.names, row.names,  
                  na.strings, nrows, skip)
```

- ▶ `col.names`: Argumento de tipo vector, opcional. Indica el nombre de las columnas.
- ▶ `row.names`: Argumento de tipo vector, opcional. Indica el nombre de las filas. También se puede indicar que una columna contiene estos nombres.
- ▶ `na.strings`: Argumento character opcional. Indica cómo se declaran los valores perdidos (Not Available).
- ▶ `nrows`: Argumento integer opcional. Indica el número máximo de filas a leer.
- ▶ `skip`: Argumento integer opcional. Indica el número de filas iniciales del archivo que no serán leídas.

Ejemplo 2

Ejecutar los siguientes códigos para leer el archivo Cap3 - INEI - 002.csv. ¿Realizan la misma acción? ¿En qué se diferencian? ¿Cuál prefieres utilizar?

```
datos2a = import("Cap3 - INEI - 002.csv")  
datos2b = read.table("Cap3 - INEI - 002.csv", TRUE, sep = ",")  
datos2c = read.csv("Cap3 - INEI - 002.csv")
```

read_csv

```
datos <- read_csv(file, col_names, na, skip, n_max)
```

- ▶ **file**: Argumento character obligatorio. Corresponde al archivo de datos a ser leído. Va entre comillas. Incluir su extensión (csv), por ejemplos "datos927.csv"
- ▶ **col_names**: Argumento opcional de múltiple tipo. Puede ser logical (TRUE o FALSE) si indica la presencia o ausencia de nombres de columnas (como header en read.table), o puede ser un vector indicando los nombres de las columnas. Por defecto toma el valor TRUE
- ▶ **na**: Argumento de tipo vector character opcional. Indica cómo se declaran los valores perdidos. Por defecto es c("", "NA").
- ▶ **skip**: Argumento numérico opcional. Indica el número de filas iniciales del archivo que no serán leídas. Por defecto es 0.
- ▶ **n_max**: Argumento numérico opcional. Indica el número máximo de filas a leer. Por defecto es infinito (Inf).

Ejercicio 2 (cont.)

Volver a leer el archivo `Cap3 - INEI - 002.csv` utilizando la función `read_csv`. Tener en cuenta que los argumentos pueden variar en nombre y no son necesariamente los mismos, además de que no son los únicos disponibles. El único argumento obligatorio es `file`.

read.csv2

```
datos <- read.csv2(file, header, sep, dec, col.names, row.names,  
                   na.strings, nrow, skip)
```

- ▶ **file**: Argumento character obligatorio. Corresponde al archivo de datos a ser leído. Va entre comillas. Incluir su extensión (txt, csv), por ejemplos "datos159.csv"
- ▶ **header**: Argumento logical opcional. TRUE si la primera línea contiene los nombres de las variables. Por defecto se asume que es TRUE.
- ▶ **sep**: Argumento character opcional. Indica el separador entre columnas. Va entre comillas, por defecto asume que es `sep = ";"` (coma).
- ▶ **dec**: Argumento character opcional. Indica el separador de la parte entera y decimal en los números, por defecto asume `dec = ","`

```
datos <- read.csv2(file, header, sep, dec, col.names, row.names,  
                  na.strings, nrows, skip)
```

- ▶ `col.names`: Argumento de tipo vector, opcional. Indica el nombre de las columnas.
- ▶ `row.names`: Argumento de tipo vector, opcional. Indica el nombre de las filas. También se puede indicar que una columna contiene estos nombres.
- ▶ `na.strings`: Argumento character opcional. Indica cómo se declaran los valores perdidos (Not Available).
- ▶ `nrows`: Argumento integer opcional. Indica el número máximo de filas a leer.
- ▶ `skip`: Argumento integer opcional. Indica el número de filas iniciales del archivo que no serán leídas.

Ejemplo 3

Ejecutar los siguientes códigos para leer el archivo Cap3 - INEI - 003.csv. ¿Realizan la misma acción? ¿En qué se diferencian? ¿Cuál prefieres utilizar?

```
datos3a = import("Cap3 - INEI - 003.csv")
datos3b = read.table("Cap3 - INEI - 003.csv", TRUE, sep = ";")
datos3c = read.csv("Cap3 - INEI - 003.csv", sep = ";")
datos3d = read.csv2("Cap3 - INEI - 003.csv")
```

read_csv2

```
datos <- read_csv2(file, col_names, na, skip, n_max)
```

- ▶ **file**: Argumento character obligatorio. Corresponde al archivo de datos a ser leído. Va entre comillas. Incluir su extensión (csv), por ejemplos "datos927.csv"
- ▶ **col_names**: Argumento opcional de múltiple tipo. Puede ser logical (TRUE o FALSE) si indica la presencia o ausencia de nombres de columnas (como `header` en `read.table`), o puede ser un vector indicando los nombres de las columnas. Por defecto toma el valor TRUE
- ▶ **na**: Argumento de tipo vector character opcional. Indica cómo se declaran los valores perdidos. Por defecto es `c("", "NA")`.
- ▶ **skip**: Argumento numérico opcional. Indica el número de filas iniciales del archivo que no serán leídas. Por defecto es 0.
- ▶ **n_max**: Argumento numérico opcional. Indica el número máximo de filas a leer. Por defecto es infinito (`Inf`).

Ejercicio 3 (cont.)

Volver a leer el archivo `Cap3 - INEI - 003.csv` utilizando la función `read_csv2`.

Tener en cuenta que los argumentos pueden variar en nombre y no son necesariamente los mismos, además de que no son los únicos disponibles. El único argumento obligatorio es `file`.

`read_xls`, `read_xlsx` y `read_excel`

Cargar el paquete `readxl`

```
library(readxl)
```

```
datos <- read_excel(path, sheet, range, col_names, na, skip, n_max)
```

- ▶ `path`: Argumento character obligatorio. Corresponde al archivo de datos de extensión `.xls` o `.xlsx` a ser leído. Va entre comillas. Incluir su extensión, por ejemplo `datos333.xlsx`.
- ▶ `sheet`: Argumento de tipo variable. Puede ser de tipo character indicando el nombre de la hoja a ser leída, o de tipo entero indicando la posición de la hoja. Es ignorado si se indica el nombre de hoja en `range`.
- ▶ `range`: Argumento character opcional. Indica el rango de celdas a ser leído. Por ejemplo `"B3:D87"` o `"Budget!B2:G14"`, donde `Budget` es el nombre de la hoja. Este argumento es más importante que `sheet`, `skip` y `n_max`.

```
datos <- read_excel(path, sheet, range, col_names, na, skip, n_max)
```

- ▶ `col_names`: Argumento opcional de múltiple tipo. Puede ser logical (TRUE o FALSE) si indica la presencia o ausencia de nombres de columnas (como `header` en `read.table`), o puede ser un vector indicando los nombres de las columnas. Por defecto toma el valor TRUE
- ▶ `na`: Argumento de tipo vector character opcional. Indica cómo se declaran los valores perdidos. Por defecto una celda en blanco es un valor perdido.
- ▶ `skip`: Argumento numérico opcional. Indica el número de filas iniciales del archivo que no serán leídas. Si hay líneas completas vacías al inicio del archivo son automáticamente ignoradas. Por defecto es 0. Es ignorado si se usa el argumento `range`.
- ▶ `n_max`: Argumento numérico opcional. Indica el número máximo de filas a leer. Si hay líneas completas vacías al final del archivo son automáticamente ignoradas. Por defecto es infinito (Inf). Es ignorado si se usa el argumento `range`.

Ejemplo 4

Ejecutar los siguientes códigos para leer los archivos Cap3 - INEI - 004.xlsx y Cap3 - INEI - 005.xls. ¿Realizan la misma acción? ¿En qué se diferencian? ¿Cuál prefieres utilizar?

```
datos4a = import("Cap3 - INEI - 004.xlsx")
datos4b = read_excel("Cap3 - INEI - 004.xlsx")
datos4c = read_xlsx("Cap3 - INEI - 004.xlsx")
datos4d = import("Cap3 - INEI - 005.xls")
datos4e = read_excel("Cap3 - INEI - 005.xls")
datos4f = read_xls("Cap3 - INEI - 005.xls")
```

Ejercicio 5

1. Leer cada una de las hojas del archivo **Cap3 - INEI - 006.xlsx** y almacenarlas en `df1`, `df_amazonas`, `df_ancash` y `df_apurimac`.
2. Extraer las 12 primeras líneas de `df_amazonas` y almacenar el resultado en `df_amazonas_12`.
3. Extraer la columna Cantidad de `df_ancash` y almacenar el resultado en `cantidad_ancash`.
4. Crear una lista conteniendo dos valores: `df_amazonas_12` y `cantidad_ancash`.

Completar la siguiente tabla:

Función	Paquete	Formato de archivo	Separador esperado (entre datos)	Separador decimal
import				
read.table				
read.csv				
read.csv2				
read.delim				
read.delim2				
read_csv				
read_csv2				
read_tsv				
read_delim				
read_excel				
read_xlsx				
read_xls				

Completar la siguiente tabla:

Argumento	utils	readr	readxl	rio
Nombre del archivo	file	file	path	file
Encabezados	header			header (csv, txt), col_names (Excel)
Separador de columnas		delim (solo con read_delim)	No aplica	sep (csv, txt)
Separador decimal		locale(decimal_mark =)	No aplica	dec(csv, txt)
Valores perdidos	na.strings		na	na.strings (csv, txt), na (Excel)
Saltar las primeras filas		skip		skip
Leer un máximo de filas			n_max	nrows (csv, txt), n_max (Excel)
Nombres de columnas	col.names	col_names		col.names (csv, txt), col_names (Excel)
Hoja de Excel	No aplica	No aplica	sheet	which / sheet
Rango de Excel	No aplica	No aplica		range